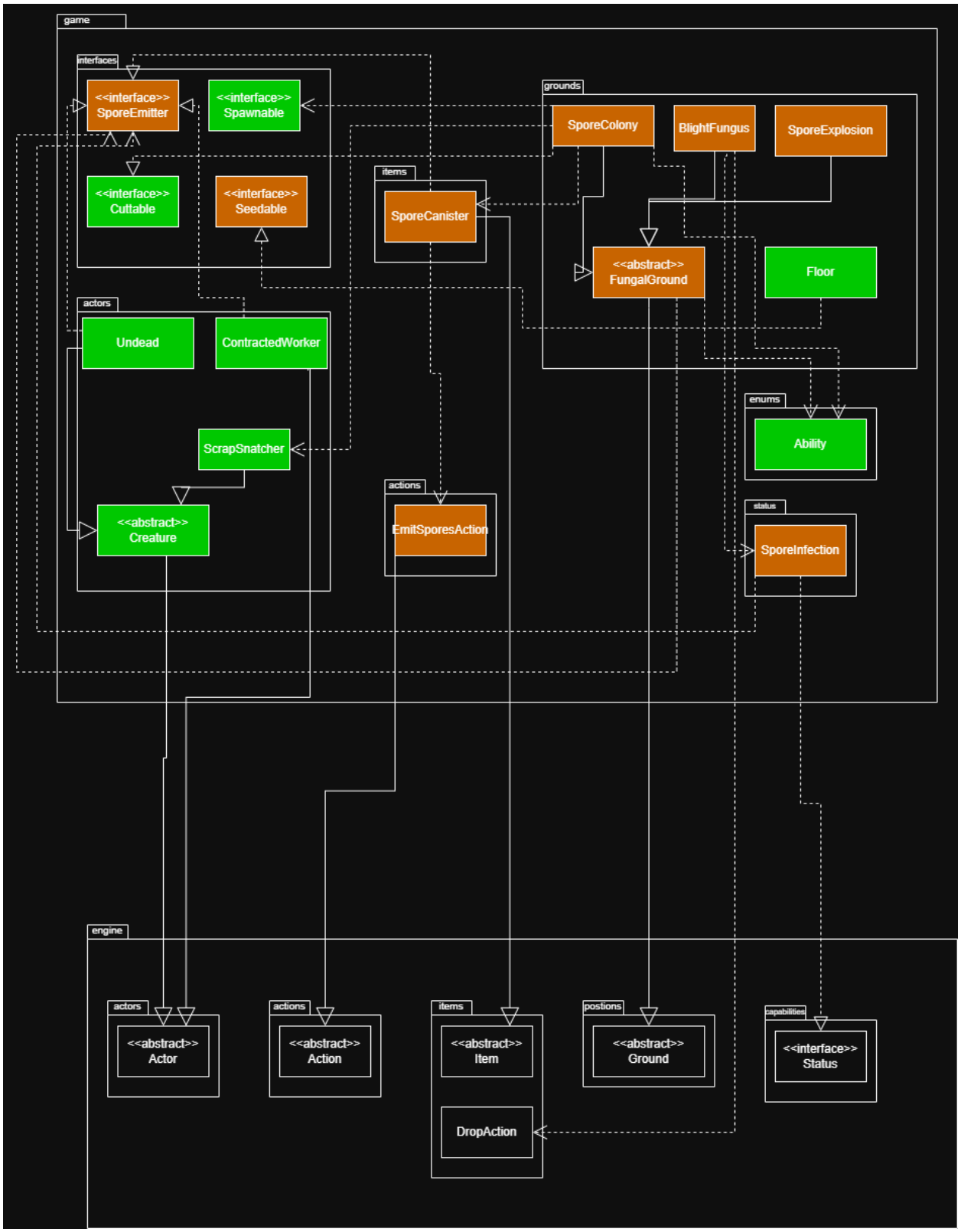
FIT2099 Assignment 3 — REQ3 Design Rationale



A. Implementation of the FungalGround Base Class

Design 1: Implement all fungal behaviour directly inside each concrete ground class (BlightFungus, SporeExplosion, SporeColony) with no shared superclass.

Pros	Cons
Each class is fully self-contained; no inheritance chain to trace when reading a single class.	**Divergent Change smell:** every future change to the core tick() skeleton (e.g. adding a "freeze frame" if the actor is stunned) must be made identically in all three classes independently, violating DRY.
	Connascence of Algorithm: all three classes must maintain the same actor-presence check, spread timer, and SporeEmitter delegation algorithm in lock-step. Any divergence silently introduces inconsistent behaviour that is hard to detect.
	No shared type for fungal ground means callers cannot use `getGroundAs(FungalGround.class)` to generically detect any fungal tile, forcing `instanceof` or repeated concrete-class checks — a classic **Inappropriate Intimacy** code smell.

Design 2 (Chosen): Create an abstract class FungalGround extending the engine Ground class, using the Template Method pattern to share the tick() skeleton while delegating per-type behaviour to abstract hooks onActorPresent() and spread().

Pros	Cons
Eliminates the Divergent Change smell: adding logic common to all fungal grounds (e.g. a "freeze frame" or immunity check) requires a single change in FungalGround.tick() rather than three. This directly satisfies DRY.	Inheritance introduces **Connascence of Inheritance** between FungalGround and its subclasses; changes to FungalGround.tick() propagate to all three concrete types simultaneously, requiring careful regression testing.
Connascence of Name is reduced to only the two abstract method signatures (onActorPresent, spread), rather than requiring every caller to know the names and semantics of each concrete class. This keeps the coupling surface minimal (KISS).	
Provides a shared polymorphic type: callers use `getGroundAs(FungalGround.class)`	

without knowing which subtype is present. For example, SporeInfection can check whether the actor's current tile is any fungal ground without being coupled to BlightFungus specifically (Open-Closed Principle — callers do not need modification when a new fungal type is added).	
Maintainability foresight: if a future requirement adds a PestFungus that deals poison damage on contact, the developer only needs to create `PestFungus extends FungalGround` and override onActorPresent() — zero changes to existing classes (Open-Closed Principle).	

Chosen: Design 2.

The Template Method pattern in FungalGround.tick() eliminates the Divergent Change smell and Connascence of Algorithm across all three concrete fungal ground types. The shared polymorphic type removes Inappropriate Intimacy from callers, and the minimal Connascence of Name on just two hook method signatures keeps the design KISS-compliant.

B. Implementation of SporeEmitter — Interface vs Abstract Class

Design 1: Create an abstract class SporeEmitter that ContractedWorker, Undead, and SporeCanister all extend.

Pros	Cons
Common utility methods (e.g. a shared helper to iterate adjacent tiles) could be inherited by all emitters, reducing duplication.	Java permits only single inheritance. ContractedWorker already extends Actor, Undead extends Creature, and SporeCanister extends Item — none of them can additionally extend SporeEmitter. This design is **structurally incompatible** with the existing hierarchy.
	Models an "is-a" relationship where only "has-a capability" is needed, violating the **Interface Segregation Principle**. Forcing all three classes into a single abstract hierarchy introduces unnecessary **Connascence of Inheritance** between otherwise unrelated entities (actors and items).
	Shotgun Surgery smell: any future change to the shared emitter base (e.g. adding a

	cooldown field) would ripple into ContractedWorker, Undead, and SporeCanister simultaneously, all of which serve very different roles in the game.

Design 2 (Chosen): Define SporeEmitter as an interface with two methods — emitSpores(Location) and getSporeType() — implemented independently by each class.

Pros	Cons
Fully compatible with the existing hierarchy: ContractedWorker (extends Actor), Undead (extends Creature), and SporeCanister (extends Item) all implement the interface without altering their inheritance chains (Interface Segregation Principle — each class only adopts the capability it needs).	Each implementing class must independently define emitSpores() and getSporeType() with no shared default, introducing a small degree of **Connascence of Name** on the method signatures across three classes.
Using `asCapability(SporeEmitter.class)` throughout the system eliminates `instanceof` checks and their associated **Connascence of Type** (callers need not know which concrete class they are dealing with). This keeps FungalGround and SporeInfection closed to modification when new emitters are added (Open-Closed Principle).	
Maintainability foresight / hypothetical example: if a new creature class HiveCreature needs to spread SporeColony instead of BlightFungus when it walks through fungal terrain, the developer only adds `implements SporeEmitter` to HiveCreature and overrides the two methods. No changes are needed in FungalGround, SporeInfection, or any existing class (Open-Closed Principle).	

Chosen: Design 2.

Defining SporeEmitter as an interface removes Connascence of Inheritance between unrelated entities, resolves the structural incompatibility with Java's single-inheritance model, and avoids the Shotgun Surgery smell. The `asCapability()` mechanism eliminates Connascence of Type on callers, keeping the system open for new emitters without modification.

C. Infection Spread Mechanism — Status-Based vs Inline Approach

Design 1: Handle the spread effect directly inside BlightFungus.tick() — after infecting an actor, BlightFungus tracks infected actors and seeds tiles beneath them each subsequent turn.

Pros	Cons
All infection logic is co-located in one class, making it easy to find.	**Feature Envy smell:** BlightFungus would need to query and track actor state (which actors are infected, where they are currently standing) that is not its responsibility — a clear violation of the Single Responsibility Principle.
	Connascence of Execution: the spread effect is tightly bound to BlightFungus's tick() execution order. The moment the actor steps off the BlightFungus tile the effect stops, even though the actor should remain contagious for 5 full turns — this produces incorrect observable behaviour.
	Not reusable: if a future SporeCloud item or PoisonVent ground should also infect actors, the spread logic cannot be reused without duplication (DRY violation).

Design 2 (Chosen): Create a SporeInfection Status (implementing the engine Status interface) that attaches to the actor, tracks a 5-turn duration, and delegates to the actor's SporeEmitter capability each turn with a 30% probability.

Pros	Cons
Responsibilities are clearly separated (Single Responsibility Principle): BlightFungus applies the status on contact; SporeInfection owns the timer and probability; the actor's emitSpores() defines what is seeded. No class "envies" another's data.	Actors that do not implement SporeEmitter (e.g. Parasite) will silently carry the status without producing a visible effect — a subtle **Connascence of Convention** that should be documented.
Connascence of Name is kept minimal: SporeInfection only depends on the SporeEmitter interface name, not on ContractedWorker or Undead directly. This means the two retrofitted actors can be renamed or replaced without modifying SporeInfection (Dependency Inversion Principle).	
The infection travels with the actor across the entire map for its full duration, correctly	

modelling an unwilling vector — the actor spreads blight to every Floor tile they walk on during those 5 turns, producing the intended mechanic.	
Hypothetical example: if a future requirement adds a ToxicVent ground that should also infect actors with blight, it only calls `actor.addStatus(new SporeInfection(5))` — zero changes to SporeInfection, ContractedWorker, or Undead (Open-Closed Principle).	

Chosen: Design 2.

The SporeInfection Status eliminates the Feature Envy smell from BlightFungus, resolves the Connascence of Execution flaw that caused premature termination of the effect, and reuses the engine's existing Status lifecycle. Connascence of Name on SporeEmitter is the only coupling surface, keeping the design consistent with DRY and the Dependency Inversion Principle.

D. SporeCanister Deployment — Fixed Type vs Supplier-Parameterised Type Selection

Design 1: SporeCanister always seeds BlightFungus when deployed. EmitSporesAction calls emitter.emitSpores() which hardcodes new BlightFungus().

Pros	Cons
Simple to implement — a single action per adjacent Floor tile, no factory pattern needed (KISS for this single case).	**Connascence of Algorithm:** EmitSporesAction and SporeCanister.emitSpores() must both agree that "BlightFungus is the seeded type", duplicating that knowledge in two places with no enforcement.
	Divergent Change smell: to later allow seeding SporeExplosion or SporeColony, both EmitSporesAction and SporeCanister.emitSpores() would need to be modified — modifying two classes for one logical change (Open-Closed violation).
	Eliminates gameplay depth: the player cannot place a SporeExplosion trap or create a new SporeColony barrier, reducing the strategic value of the canister.

Design 2 (Chosen): Parameterise EmitSporesAction with a Supplier<FungalGround> factory and a display name. SporeCanister.allowableActions() generates three actions per adjacent Floor tile (one per fungal type). The AoE burst (25% chance) seeds the same chosen type on all surrounding Floor tiles.

Pros	Cons
Connascence of Name on the concrete ground class names (BlightFungus, SporeExplosion, SporeColony) is confined entirely to SporeCanister.allowableActions() — the single point of knowledge. EmitSporesAction depends only on the `Supplier<FungalGround>` abstraction (Dependency Inversion Principle).	Generates up to three menu entries per adjacent Floor tile, increasing menu length when the player holds a SporeCanister near multiple Floor tiles.
Eliminates the Divergent Change smell: adding a future FrostMold ground type only requires one new line in SporeCanister.allowableActions() — `new EmitSporesAction(FrostMold::new, "FrostMold", target, dir)` — with zero changes to EmitSporesAction itself (Open-Closed Principle).	
The AoE burst seeds the same type as the player chose, keeping **Connascence of Algorithm** self-contained within one method (EmitSporesAction.execute()) rather than split across two classes.	
KISS maintained: the design is still simple — each action is a plain object with a factory and a name, and calling `groundFactory.get()` is the only mechanism needed to produce any fungal type, however many are added.	

Chosen: Design 2.

The `Supplier<FungalGround>` parameter confines Connascence of Name on concrete ground types to a single location, eliminating the Divergent Change smell that would arise from Design 1. The AoE burst consolidates Connascence of Algorithm within one method, and KISS is preserved — the factory call is the only mechanism needed regardless of how many fungal types are added.

Final Design & SOLID / Connascence Summary

The final design adopts Design 2 for all four decisions to implement the Fungal Bloom System. Each key design choice is summarised below:

FungalGround (abstract Ground subclass): BlightFungus, SporeExplosion, and SporeColony extend FungalGround. The Template Method pattern in tick() enforces a consistent execution order (actor-contact → SporeEmitter trigger → passive spread), eliminating Divergent Change and Connascence of Algorithm across all subclasses. Any future fungal type inherits this structure automatically — zero changes to existing classes (Open-Closed Principle, DRY).

SporeEmitter (interface): Implemented by SporeCanister, ContractedWorker, and Undead — three classes from entirely different inheritance chains. Defining emission as an interface avoids structural incompatibility, removes Connascence of Inheritance between unrelated entities, and enables `asCapability()` to resolve emitters at runtime without `instanceof`, keeping type-checking closed to modification (Open-Closed Principle, Interface Segregation Principle).

SporeInfection (Status): Isolates the per-actor spread behaviour into the engine's existing Status lifecycle, eliminating Feature Envy from BlightFungus and resolving Connascence of Execution. The infection travels with the actor for its full 5-turn duration. By depending only on the SporeEmitter interface, SporeInfection exhibits only Connascence of Name on the interface — not on any concrete actor class — minimising coupling (Dependency Inversion Principle, Single Responsibility Principle).

EmitSporesAction (Supplier-parameterised): Confines all knowledge of concrete ground class names to a single location in `SporeCanister.allowableActions()`, eliminating Connascence of Algorithm and the Divergent Change smell. The Seedable marker interface on Floor provides a type-safe seeding check, removing all `instanceof Floor` calls and keeping the system consistent with the project-wide no-instanceof policy (Open-Closed Principle, KISS).

SporeColony implements Cuttable (REQ1 integration): SporeColony reuses the existing Cuttable contract from REQ1, dropping a SporeCanister and converting to a Floor when cut with the Plasma Cutter. The REQ1 cutting system gains SporeColony support without a single line of change to CutAction or ContractedWorker — demonstrating the Open-Closed Principle at the inter-requirement level.